1 dubbo的基础知识

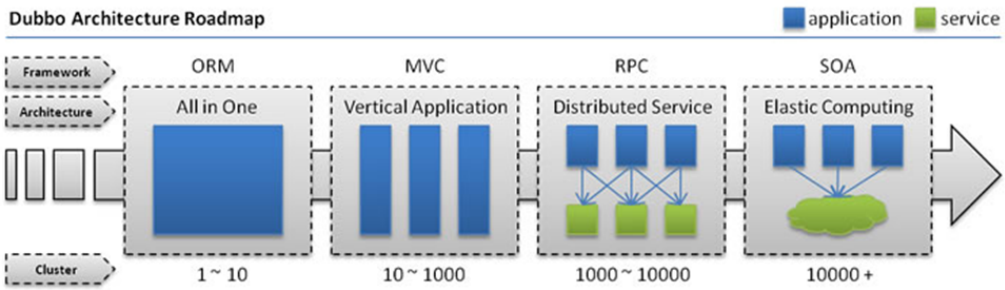
1.1 分布式基础理论

随着互联网的发展，网站应用的规模不断扩大，常规的垂直应用架构已无法应对，分布式服务架构以及流动计算架构势在必行，亟需**一个治理系统**确保架构有条不紊的演进。

《分布式系统原理与范型》定义："分布式系统是若干独立计算机的集合，这些计算机对于用户来说就像单个相关系统"。分布式系统（distributed system）是建立在网络之上的软件系统。

1.2 系统架构的发展演变

这是dubbo官网的一张图，介绍了系统架构的发展演变：



1.2.1 单体架构的应用

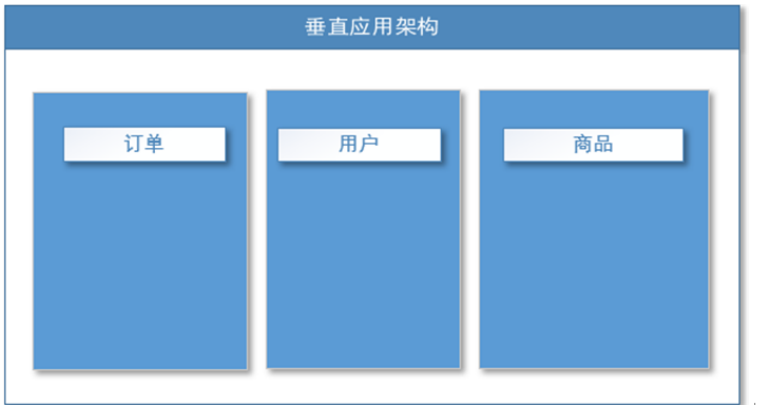
当网站流量很小时，只需一个应用，将所有功能都部署在一起，以减少部署节点和成本。此时，用于简化增删改查工作量的数据访问框架(ORM)是关键。

适用于小型网站，小型管理系统，将所有功能都部署到一个功能里，简单易用。

缺点：1、性能扩展比较难　2、协同开发问题　3、不利于升级维护

1.2.2 垂直应用架构

当访问量逐渐增大，单一应用增加机器带来的加速度越来越小，将应用拆成互不相干的几个应用，以提升效率。此时，用于加速前端页面开发的Web框架(MVC)是关键。

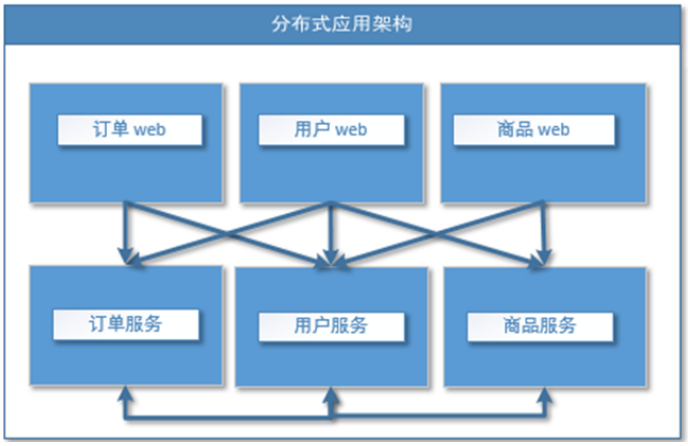


通过切分业务来实现各个模块独立部署，降低了维护和部署的难度，团队各司其职更易管理，性能扩展也更方便，更有针对性。

缺点：公用模块无法重复利用，开发性的浪费

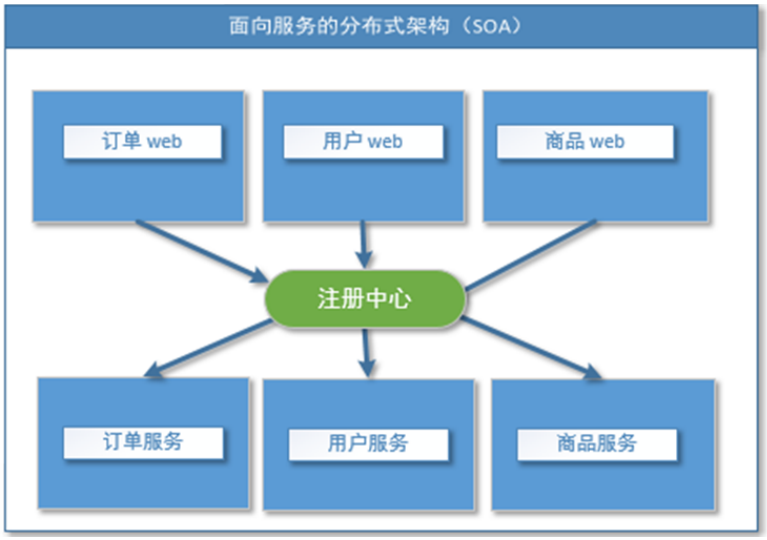
1.2.3 分布式服务架构

当垂直应用越来越多，应用之间交互不可避免，将核心业务抽取出来，作为独立的服务，逐渐形成稳定的服务中心，使前端应用能更快速的响应多变的市场需求。此时，用于提高业务复用及整合的**分布式服务框架(RPC)**是关键。



1.2.4 流动计算架构

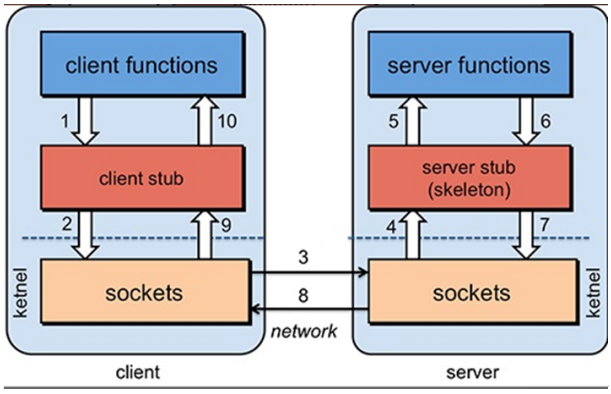
当服务越来越多，容量的评估，小服务资源的浪费等问题逐渐显现，此时需增加一个调度中心基于访问压力实时管理集群容量，提高集群利用率。此时，用于**提高机器利用率的资源调度和治理中心(SOA)[Service Oriented Architecture]是关键。**



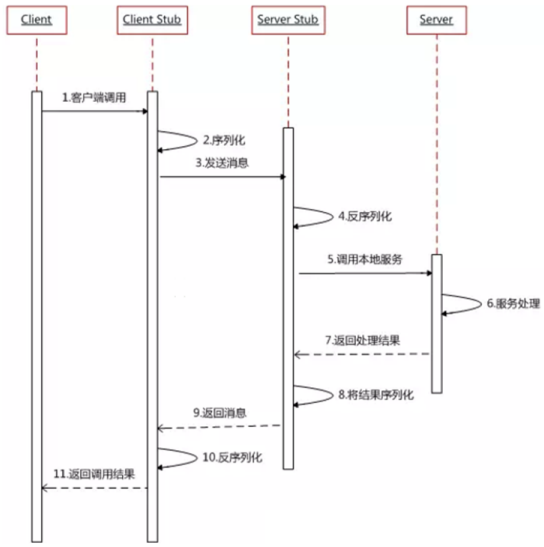
1.3 RPC

RPC【Remote Procedure Call】是指远程过程调用，是一种进程间通信方式，他是一种技术的思想，而不是规范。它允许程序调用另一个地址空间（通常是共享网络的另一台机器上）的过程或函数，而不用程序员显式编码这个远程调用的细节。即程序员无论是调用本地的还是远程的函数，本质上编写的调用代码基本相同。

RPC的基本原理：



我们进一步细化得到如下的图：



RPC两个核心模块：通讯(客户端和服务器之间快速建立连接)，序列化。（数据序列化和反序列化的效率）

1.4 dubbo的核心概念

1.4.1 dubbo的简介

Apache Dubbo (incubating) |ˈdʌbəʊ|是一款高性能、轻量级的开源Java RPC框架，它提供了三大核心能力：面向接口的远程方法调用，智能容错和负载均衡，以及服务自动注册和发现。

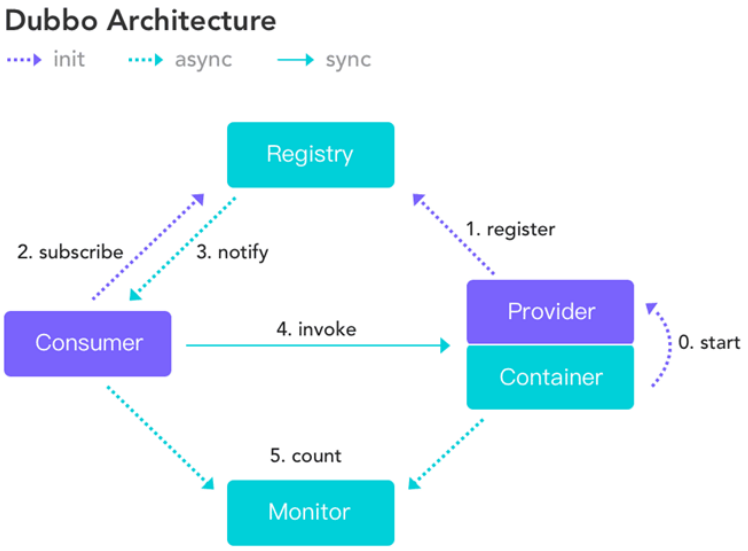
官网：http://dubbo.apache.org/

Dubbo的发展历程：

> Dubbo自2008 年底在阿里内部开始规划调用，2009 年初开发 1.0 版本；2010 年 04 月在 1.0 的版本之上进行了重构，发布了 2.0 版本；2011 年 10 月阿里宣布将 Dubbo 开源，开源的第一个版本为版本 dubbo-2.0.7。
>
> Dubbo 开源之后，框架发展比较迅速，几乎两三个月会发布一个版本，于 2012 年 3 月 14 号发布版本 dubbo-2.1.0。随后又进入另一个快速发展期，版本发布频繁，几乎每一个月会发布好几次。直到 2013 年 3 月 17 号发布了 dubbo-2.4.10，版本陷入停滞；2014 年 10 月 30 号发布版本 dubbo-2.4.11，修复了一个小 Bug，版本又陷入漫长的停滞到现在。
>
> 2014 年的 10 月 20 号，当当网 Fork 了阿里的一个 Dubbo 版本开始维护，并命名为 dubbox-2.8.0。值得注意的是，当当网扩展 Dubbo 服务框架支持 REST 风格远程调用，并且跟随着 ZooKeepe 和 Spring 升级了对应的版本。之后 Dubbox 一直在小版本维护，2015 年 3 月 31 号发布了最后一个版本 dubbox-2.8.4。

> 伴随着Dubbo和Dubbox的停止维护，让人们开始怀疑Dubbo的未来发展，除了本身存在的bug之外，Dubbo也越来越难以追上其所依赖的Jdk、Spring、Zookeeper、Zkclient等等组件更新的脚步，将要慢慢被淘汰。
>
> 好在，停止维护将近三年，随着SprigCloud在抢占分布式开发框架市场上大红大紫，Dubbo又开始了更新的脚步。2017 年 9 月，阿里发布了 dubbo-2.5.4/5 版本；2017 年 10 月，阿里发布了 dubbo-2.5.6 版本；2017 年 11 月，阿里发布了 dubbo-2.5.7····
>
> 在18年1月，Dubbo与Dubbox合并，完成 dubbo-2.6.0 版本的升级。2月15号，Dubbo将自己开源贡献给了Apache组织。
>
> Dubbo 被证实能很好的满足企业的大规模微服务实践，并且能有效降低微服务建设的开发与管理成本，不论是阿里巴巴还是工商银行、中国平安、携程、海尔等社区用户，它们都通过多年的大规模生产环境流量对 Dubbo 的稳定性与性能进行了充分验证。后来 Dubbo 在很多大企业内部衍生出了独立版本。自云原生概念推广以来，各大厂商都开始拥抱开源标准实现，阿里巴巴将其内部 HSF 系统与开源社区 Dubbo 相融合，与社区一同推出了云原生时代的 Dubbo3 架构，截止 2022 年双十一结束，**Dubbo3 已经在阿里巴巴内部广泛落地，实现了老版本 HSF2 框架升级，包括电商核心、阿里云等核心系统已经全面运行在 Dubbo3 之上。**

1.4.2 dubbo的基本概念



> **服务提供者（Provider）**：暴露服务的服务提供方，服务提供者在启动时，向注册中心注册自己提供的服务。
>
> **服务消费者（Consumer）**:调用远程服务的服务消费方，服务消费者在启动时，向注册中心订阅自己所需的服务，服务消费者，从提供者地址列表中，基于软负载均衡算法，选一台提供者进行调用，如果调用失败，再选另一台调用。

> **注册中心（Registry）**：注册中心返回服务提供者地址列表给消费者，如果有变更，注册中心将基于长连接推送变更数据给消费者
>
> **监控中心（Monitor）**：服务消费者和提供者，在内存中累计调用次数和调用时间，定时每分钟发送一次统计数据到监控中心

调用关系说明:

> - 服务容器负责启动，加载，运行服务提供者。
> - 服务提供者在启动时，向注册中心注册自己提供的服务。
> - 服务消费者在启动时，向注册中心订阅自己所需的服务。
> - 注册中心返回服务提供者地址列表给消费者，如果有变更，注册中心将基于长连接推送变更数据给消费者。
> - 服务消费者，从提供者地址列表中，基于软负载均衡算法，选一台提供者进行调用，如果调用失败，再选另一台调用。
> - 服务消费者和提供者，在内存中累计调用次数和调用时间，定时每分钟发送一次统计数据到监控中心。

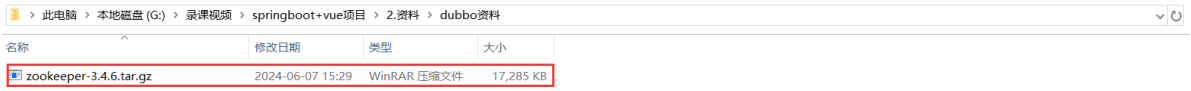
1.5 dubbo环境的搭建

1.5.1 在windows环境下面安装zookeeper

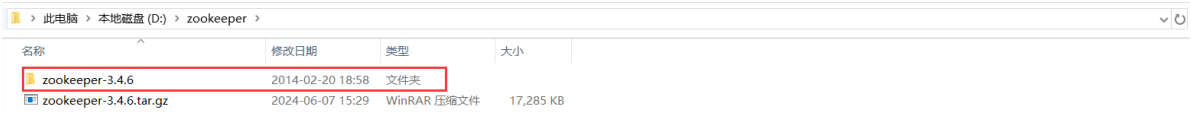
第一步：下载zookeeper

下载地址：https://archive.apache.org/dist/zookeeper/zookeeper-3.4.6/

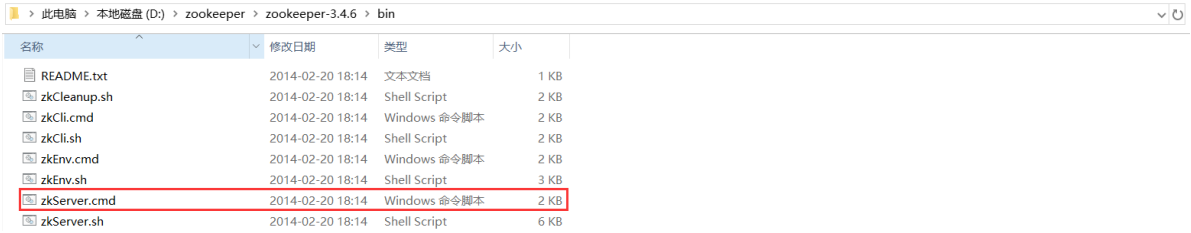
下载之后得到一个压缩包：



第二步：解压zookeeper



解压运行zkServer.cmd，**初次运行会报错，没有zoo.cfg配置文件**



启动zookeeper：

```
D:\zookeeper\zookeeper-3.4.6\bin>zkServer.cmd

D:\zookeeper\zookeeper-3.4.6\bin>java "-Dzookeeper.log.dir=D:\zookeeper\zookeeper-3.4.6\bin\.." "-Dzookeeper.root.logger=INFO,CONSOLE" -cp "D:\zookeeper\zookeep
er-3.4.6\bin\..\build\classes;D:\zookeeper\zookeeper-3.4.6\bin\..\build\lib\*;D:\zookeeper\zookeeper-3.4.6\bin\..\*;D:\zookeeper\zookeeper-3.4.6\bin\..\lib\*;D:
\zookeeper\zookeeper-3.4.6\bin\..\conf" org.apache.zookeeper.server.quorum.QuorumPeerMain "D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg"
2024-06-07 15:36:45,614 [myid:] - INFO  [main:QuorumPeerConfig@103] - Reading configuration from: D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg
2024-06-07 15:36:45,617 [myid:] - ERROR [main:QuorumPeerMain@85] - Invalid config, exiting abnormally
org.apache.zookeeper.server.quorum.QuorumPeerConfig$ConfigException: Error processing D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg
        at org.apache.zookeeper.server.quorum.QuorumPeerConfig.parse(QuorumPeerConfig.java:123)
        at org.apache.zookeeper.server.quorum.QuorumPeerMain.initializeAndRun(QuorumPeerMain.java:101)
        at org.apache.zookeeper.server.quorum.QuorumPeerMain.main(QuorumPeerMain.java:78)
Caused by: java.lang.IllegalArgumentException: D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg file is missing
        at org.apache.zookeeper.server.quorum.QuorumPeerConfig.parse(QuorumPeerConfig.java:107)
        ... 2 more
Invalid config, exiting abnormally
```

zoo.cfg文件不存在

第三步：修改zoo.cfg配置文件

将conf下的zoo_sample.cfg复制一份改名为zoo.cfg即可。并对zoo.cfg做如下修改：

```
# The number of milliseconds of each tick
tickTime=2000
# The number of ticks that the initial
# synchronization phase can take
initLimit=10
# The number of ticks that can pass between
# sending a request and getting an acknowledgement
syncLimit=5
# the directory where the snapshot is stored.
# do not use /tmp for storage, /tmp here is just
# example sakes.
dataDir=../data
# the port at which the clients will connect
clientPort=2181
# the maximum number of client connections.
# increase this if you need to handle more clients
```

> 此电脑 > 本地磁盘 (D:) > zookeeper > zookeeper-3.4.6

名称	修改日期	类型	大小
bin	2014-02-20 18:48	文件夹	
conf	2024-06-07 15:38	文件夹	
contrib	2014-02-20 18:14	文件夹	
data	2024-06-07 15:38	文件夹	
dist-maven	2014-02-20 19:05	文件夹	
docs	2014-02-20 18:48	文件夹	
lib	2014-02-20 18:48	文件夹	
recipes	2014-02-20 18:14	文件夹	
src	2014-02-20 18:48	文件夹	

第三步：再次启动

```
D:\zookeeper\zookeeper-3.4.6\bin>zkServer.cmd

D:\zookeeper\zookeeper-3.4.6\bin>java "-Dzookeeper.log.dir=D:\zookeeper\zookeeper-3.4.6\bin\.." "-Dzookeeper.root.logger=INFO,CONSOLE" -cp "D:\zookeeper\zookeeper-3.4.6\bin\..\build\classes
;D:\zookeeper\zookeeper-3.4.6\bin\..\build\lib\*;D:\zookeeper\zookeeper-3.4.6\bin\..\*;D:\zookeeper\zookeeper-3.4.6\bin\..\lib\*;D:\zookeeper\zookeeper-3.4.6\bin\..\conf" org.apache.zookee
er.server.quorum.QuorumPeerMain "D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg"
2024-06-07 15:42:21,139 [myid:] - INFO  [main:QuorumPeerConfig@103] - Reading configuration from: D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg
2024-06-07 15:42:21,158 [myid:] - INFO  [main:DatadirCleanupManager@78] - autopurge.snapRetainCount set to 3
2024-06-07 15:42:21,159 [myid:] - INFO  [main:DatadirCleanupManager@79] - autopurge.purgeInterval set to 0
2024-06-07 15:42:21,159 [myid:] - INFO  [main:DatadirCleanupManager@101] - Purge task is not scheduled.
2024-06-07 15:42:21,161 [myid:] - WARN  [main:QuorumPeerMain@113] - Either no config or no quorum defined in config, running  in standalone mode
2024-06-07 15:42:21,257 [myid:] - INFO  [main:QuorumPeerConfig@103] - Reading configuration from: D:\zookeeper\zookeeper-3.4.6\bin\..\conf\zoo.cfg
2024-06-07 15:42:21,258 [myid:] - INFO  [main:ZooKeeperServerMain@95] - Starting server
2024-06-07 15:42:21,294 [myid:] - INFO  [main:Environment@100] - Server environment:zookeeper.version=3.4.6-1569965, built on 02/20/2014 09:09 GMT
2024-06-07 15:42:21,294 [myid:] - INFO  [main:Environment@100] - Server environment:host.name=DESKTOP-EKIOP48
2024-06-07 15:42:21,295 [myid:] - INFO  [main:Environment@100] - Server environment:java.version=1.8.0_131
2024-06-07 15:42:21,295 [myid:] - INFO  [main:Environment@100] - Server environment:java.vendor=Oracle Corporation
2024-06-07 15:42:21,295 [myid:] - INFO  [main:Environment@100] - Server environment:java.home=C:\Program Files\Java\jdk1.8.0_131\jre
2024-06-07 15:42:21,295 [myid:] - INFO  [main:Environment@100] - Server environment:java.class.path=D:\zookeeper\zookeeper-3.4.6\bin\..\build\classes;D:\zookeeper\zookeeper-3.4.6\bin\..\bui
ld\lib\*;D:\zookeeper\zookeeper-3.4.6\bin\..\zookeeper-3.4.6.jar;D:\zookeeper\zookeeper-3.4.6\bin\..\lib\jline-0.9.94.jar;D:\zookeeper\zookeeper-3.4.6\bin\..\lib\log4j-1.2.16.jar;D:\zookeep
er\zookeeper-3.4.6\bin\..\lib\netty-3.7.0.Final.jar;D:\zookeeper\zookeeper-3.4.6\bin\..\lib\slf4j-api-1.6.1.jar;D:\zookeeper\zookeeper-3.4.6\bin\..\lib\slf4j-log4j12-1.6.1.jar;D:\zookeeper\
zookeeper-3.4.6\bin\..\conf
2024-06-07 15:42:21,296 [myid:] - INFO  [main:Environment@100] - Server environment:java.library.path=C:\Program Files\Java\jdk1.8.0_131\bin;C:\Windows\Sun\Java\bin;C:\Windows\system32;C:\W
indows;C:\Program Files\Git\cmd;C:\Program Files\Java\jdk1.8.0_131\bin;C:\Program Files\Java\jdk1.8.0_131\jre\bin;D:\app\kriss\product\11.2.0\dbhome_1\bin;C:\Windows\system32;C:\Windows;C:\
Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\Intel\Intel(R) Management Engine Components\DAL;C:\Program Files\Intel\
Intel(R) Management Engine Components\DAL;C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files\NVIDIA Corporation\NVIDIA NGX;C:\Program Files\Intel\WiFi\bin\;C:\Program F
iles\Common Files\Intel\WirelessCommon\;C:\Program Files\MySQL\MySQL Server 5.5\bin;D:\apache-maven-3.5.2\bin;C:\Program Files\TortoiseGit\bin;C:\Program Files\erl-23.0\bin;D:\svn\bin;D:\sv
n-client\bin;D:\rocketmq\rocketmq-all-4.5.0-bin-release\bin;D:\node_nvm\nvm;D:\node_nvm\nodejs;D:\gradle-4.10.3\bin;C:\Program Files\nodejs\;C:\Program Files\MySQL\MySQL Shell 8.0\bin\;C:\U
sers\kriss\AppData\Local\Programs\Python\Python310\Scripts\;C:\Users\kriss\AppData\Local\Programs\Python\Python310\;C:\Users\kriss\AppData\Local\Microsoft\WindowsApps;D:\pyCharm\PyCharm 202
2.2\bin;;D:\vscode\Microsoft VS Code\bin;C:\Users\kriss\AppData\Roaming\npm;D:\ffmpeg-n5.0-latest-win64-lgpl-shared-5.0\bin;;.
2024-06-07 15:42:21,297 [myid:] - INFO  [main:Environment@100] - Server environment:java.io.tmpdir=C:\Users\kriss\AppData\Local\Temp\
2024-06-07 15:42:21,298 [myid:] - INFO  [main:Environment@100] - Server environment:java.compiler=<NA>
2024-06-07 15:42:21,300 [myid:] - INFO  [main:Environment@100] - Server environment:os.name=Windows 10
2024-06-07 15:42:21,300 [myid:] - INFO  [main:Environment@100] - Server environment:os.arch=amd64
2024-06-07 15:42:21,301 [myid:] - INFO  [main:Environment@100] - Server environment:os.version=10.0
2024-06-07 15:42:21,302 [myid:] - INFO  [main:Environment@100] - Server environment:user.name=kriss
2024-06-07 15:42:21,302 [myid:] - INFO  [main:Environment@100] - Server environment:user.home=C:\Users\kriss
2024-06-07 15:42:21,303 [myid:] - INFO  [main:Environment@100] - Server environment:user.dir=D:\zookeeper\zookeeper-3.4.6\bin
2024-06-07 15:42:21,308 [myid:] - INFO  [main:ZooKeeperServer@755] - tickTime set to 2000
2024-06-07 15:42:21,309 [myid:] - INFO  [main:ZooKeeperServer@764] - minSessionTimeout set to -1
2024-06-07 15:42:21,309 [myid:] - INFO  [main:ZooKeeperServer@773] - maxSessionTimeout set to -1
2024-06-07 15:42:21,828 [myid:] - INFO  [main:NIOServerCnxnFactory@94] - binding to port 0.0.0.0/0.0.0.0:2181
```

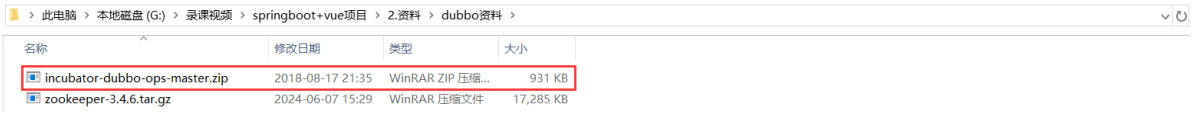
1.5.2 在windows环境下安装dubbo-admin管理控制台

dubbo本身并不是一个服务软件。它其实就是一个jar包能够帮你的java程序连接到zookeeper，并利用zookeeper消费、提供服务。所以你不用在Linux上启动什么dubbo服务。

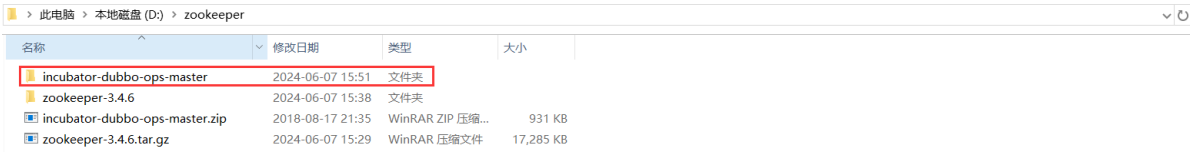
但是为了让用户更好的管理监控众多的dubbo服务，官方提供了一个可视化的监控程序，不过这个监控即使不装也不影响使用。

第一步：下载dubbo-admin

下载地址：https://github.com/apache/incubator-dubbo-ops



第二步：解压并进入目录，修改dubbo-admin配置



修改dubbo-admin的配置文件：

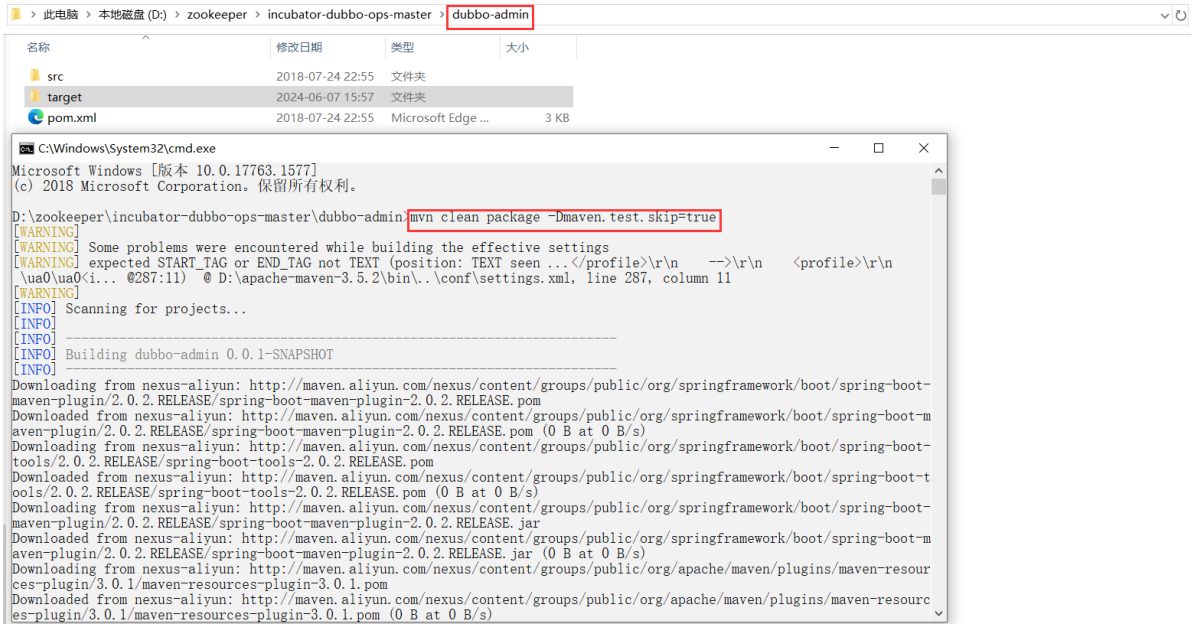
修改application.properties:

```
server.port=7001
spring.velocity.cache=false
spring.velocity.charset=UTF-8
spring.velocity.layout-url=/templates/default.vm
spring.messages.fallback-to-system-locale=false
spring.messages.basename=i18n/message
spring.root.password=root
spring.guest.password=guest

dubbo.registry.address=zookeeper://127.0.0.1:2181
```

第三步：打包dubbo-admin

执行命令：mvn clean package -Dmaven.test.skip=true



打包成功后我们会得到一个jar包：

第四步：运行dubbo-admin

使用命令：java -jar dubbo-admin-0.0.1-SNAPSHOT.jar 运行：

```
2024-06-07 16:01:58.257  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/governance/consumers/{id}/notified]}" onto public java.lang.String com.alibab
a.dubboadmin.web.mvc.governance.ConsumersController.notified(java.lang.Long, javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse, org.springframework.ui.Model)
2024-06-07 16:01:58.258  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/governance/consumers/{id}/overrided]}" onto public java.lang.String com.aliba
ba.dubboadmin.web.mvc.governance.ConsumersController.overrided(java.lang.Long, javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse, org.springframework.ui.Model)
2024-06-07 16:01:58.259  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/governance/consumers/{ids}/onlyforbid]}" onto public java.lang.String com.ali
baba.dubboadmin.web.mvc.governance.ConsumersController.onlyforbid(java.lang.Long[], javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse, org.springframework.ui.Model)
 throws java.lang.Exception
2024-06-07 16:01:58.259  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/governance/consumers/allshield]}" onto public java.lang.String com.alibaba.du
bboadmin.web.mvc.governance.ConsumersController.allshield(java.lang.String, javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse, org.springframework.ui.Model) throws
java.lang.Exception
2024-06-07 16:01:58.260  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/governance/consumers/alltolerant]}" onto public java.lang.String com.alibaba.
dubboadmin.web.mvc.governance.ConsumersController.alltolerant(java.lang.String, javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse, org.springframework.ui.Model) thr
ows java.lang.Exception
2024-06-07 16:01:58.261  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/sysinfo/dumps]}" onto public java.lang.String com.alibaba.dubboadmin.web.mvc.
sysinfo.DumpsController.index(javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse, org.springframework.ui.Model)
2024-06-07 16:01:58.264  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/error]}" onto public org.springframework.http.ResponseEntity<java.util.Map<ja
va.lang.String, java.lang.Object>> org.springframework.boot.autoconfigure.web.servlet.error.BasicErrorController.error(javax.servlet.http.HttpServletRequest)
2024-06-07 16:01:58.267  INFO 20420 --- [           main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped "{[/error],produces=[text/html]}" onto public org.springframework.web.servlet.Mod
elAndView org.springframework.boot.autoconfigure.web.servlet.error.BasicErrorController.errorHtml(javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse)
2024-06-07 16:01:58.303  INFO 20420 --- [           main] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/webjars/**] onto handler of type [class org.springframework.web.servle
t.resource.ResourceHttpRequestHandler]
2024-06-07 16:01:58.303  INFO 20420 --- [           main] o.s.w.s.handler.SimpleUrlHandlerMapping  : Mapped URL path [/**] onto handler of type [class org.springframework.web.servlet.resour
ce.ResourceHttpRequestHandler]
2024-06-07 16:01:58.513  INFO 20420 --- [           main] o.s.j.e.a.AnnotationMBeanExporter        : Registering beans for JMX exposure on startup
2024-06-07 16:01:58.558  INFO 20420 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 7001 (http) with context path ''
2024-06-07 16:01:58.567  INFO 20420 --- [           main] c.a.dubboadmin.DubboAdminApplication     : Started DubboAdminApplication in 4.692 seconds (JVM running for 5.2)
 INFO dubboadmin.SpringUtil -  [DUBBO] set applicationcontext, dubbo version: 2.6.2, current host: 192.168.10.1
```

使用7001端口进行访问，默认用户名和密码都是root：

1.6 dubbo入门案例的搭建

需求：搭建一个服务提供方、服务消费方。实现服务消费方远程调用服务提供方，并向浏览器返回一个字符串。

1.6.1 创建父工程dubbo-project

创建父工程，并引入如下依赖：

```
<properties>
    <spring.version>4.2.4.RELEASE</spring.version>
</properties>
<dependencies>
    <!-- Spring -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>${spring.version}</version>
```

```
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-beans</artifactId>
        <version>${spring.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>
        <version>${spring.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>${spring.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-aspects</artifactId>
        <version>${spring.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jms</artifactId>
        <version>${spring.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context-support</artifactId>
        <version>${spring.version}</version>
    </dependency>
    <!-- dubbo依赖 -->
    <dependency>
        <groupId>com.alibaba</groupId>
        <artifactId>dubbo</artifactId>
        <version>2.8.4</version>
    </dependency>
    <dependency>
        <groupId>org.apache.zookeeper</groupId>
        <artifactId>zookeeper</artifactId>
        <version>3.4.6</version>
    </dependency>
    <dependency>
        <groupId>com.github.sgroschupf</groupId>
        <artifactId>zkclient</artifactId>
```

```
        <version>0.1</version>
    </dependency>
    <dependency>
        <groupId>javassist</groupId>
        <artifactId>javassist</artifactId>
        <version>3.11.0.GA</version>
    </dependency>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.11</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

1.6.2 定义服务提供方

创建服务提供方dubbo-service-provider。

1.6.2.1 创建服务接口

```
public interface UserService {
    public String getName();
}
```

并创建服务接口的实现类：

```
import com.alibaba.dubbo.config.annotation.Service;
import com.xq.dubbox.service.UserService;
@Service
public class UserServiceImpl implements UserService {
    @Override
    public String getName() {
        return "hello,Dubbox.......";
    }
}
```

注意：@Service的注解一定要是dubbo提供的注解！

1.6.2.2 创建服务提供方的配置文件

创建配置文件：applicationContext-service.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:dubbo="http://code.alibabatech.com/schema/dubbo"

xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://code.alibabatech.com/schema/dubbo
http://code.alibabatech.com/schema/dubbo/dubbo.xsd">

    <dubbo:application name="dubbo-service-provider"/>
    <dubbo:registry address="zookeeper://127.0.0.1:2181"/>
    <dubbo:annotation package="com.xq.service.impl" />

</beans>
```

1.6.2.3 配置web.xml

```
<!DOCTYPE web-app PUBLIC
 "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
 "http://java.sun.com/dtd/web-app_2_3.dtd" >

<web-app>
  <display-name>Archetype Created Web Application</display-name>
  <!-- 加载spring容器 -->
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>classpath:applicationContext*.xml</param-value>
  </context-param>
  <listener>
    <listener-
class>org.springframework.web.context.ContextLoaderListener</list
ener-class>
  </listener>

</web-app>
```

1.6.2.4 在pom.xml里面配置tomcat7插件

在服务提供方的pom.xml里面定义：

```
<build>
    <finalName>service-provider</finalName>
    <plugins>
        <plugin>
```

```
            <groupId>org.apache.tomcat.maven</groupId>
            <artifactId>tomcat7-maven-plugin</artifactId>
            <version>2.1</version>
            <configuration>
                <port>8081</port>
                <server>tomcat7</server>
                <path>/</path>
            </configuration>
        </plugin>
    </plugins>
</build>
```

1.6.3 创建服务消费方

创建服务提供方dubbo-service-consumer。

1.6.3.1 拷贝服务提供方服务接口

将dubbo_service_provider里面的接口及所在的包拷贝至dubbo_service_consumer里面来

```
package com.xq.dubbox.service;

public interface UserService {
    public String getName();
}
```

1.6.3.2 定义controller

```
package com.xq.controller;

import com.alibaba.dubbo.config.annotation.Reference;
import com.xq.UserService;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller
@RequestMapping("/user")
public class UserController {

   @Reference //使用dubbo提供的注解
   UserService userService;

    @RequestMapping("/showName")
```

```
    @ResponseBody
    public String showName() {
        return userService.getName();
    }
}
```

注意：这里使用@Reference注解，不能使用@Autowired。

1.6.3.3 配置springmvc.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:dubbo="http://code.alibabatech.com/schema/dubbo"
       xmlns:mvc="http://www.springframework.org/schema/mvc"

xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://code.alibabatech.com/schema/dubbo
http://code.alibabatech.com/schema/dubbo/dubbo.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <mvc:annotation-driven >
        <mvc:message-converters register-defaults="false">
            <bean
class="org.springframework.http.converter.StringHttpMessageConver
ter">
                <constructor-arg value="UTF-8" />
            </bean>
        </mvc:message-converters>
    </mvc:annotation-driven>
    <dubbo:application name="dubbo-service-consumer" />
    <dubbo:registry address="zookeeper://127.0.0.1:2181"/>
    <dubbo:annotation package="com.xq.controller" />
</beans>
```

1.6.3.4 定义web.xml

```
<!DOCTYPE web-app PUBLIC
 "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
 "http://java.sun.com/dtd/web-app_2_3.dtd" >

<web-app>
  <display-name>Archetype Created Web Application</display-name>
```

```
  <!-- 解决post乱码 -->
  <filter>
    <filter-name>CharacterEncodingFilter</filter-name>
    <filter-
class>org.springframework.web.filter.CharacterEncodingFilter</fil
ter-class>
    <init-param>
      <param-name>encoding</param-name>
      <param-value>utf-8</param-value>
    </init-param>
    <init-param>
      <param-name>forceEncoding</param-name>
      <param-value>true</param-value>
    </init-param>
  </filter>
  <filter-mapping>
    <filter-name>CharacterEncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>

  <servlet>
    <servlet-name>springmvc</servlet-name>
    <servlet-
class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
    <!-- 指定加载的配置文件 ，通过参数contextConfigLocation加载-->
    <init-param>
      <param-name>contextConfigLocation</param-name>
      <param-value>classpath:springmvc.xml</param-value>
    </init-param>
  </servlet>

  <servlet-mapping>
    <servlet-name>springmvc</servlet-name>
    <url-pattern>*.do</url-pattern>
  </servlet-mapping>
</web-app>
```

1.6.3.4 配置tomcat7插件

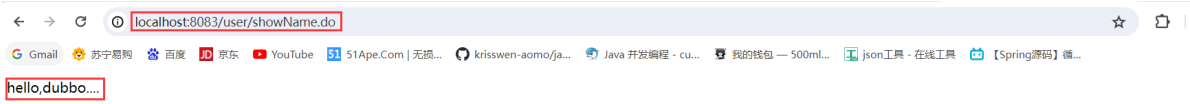
在服务消费方的pom.xml里面配置tomcat7插件

```
<build>
    <finalName>dubbo-service-consumer</finalName>
    <plugins>
```

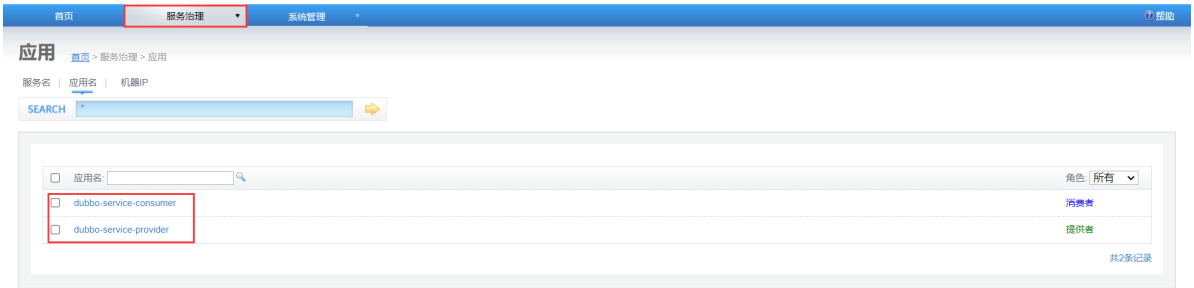
```
        <plugin>
            <groupId>org.apache.tomcat.maven</groupId>
            <artifactId>tomcat7-maven-plugin</artifactId>
            <version>2.1</version>
            <configuration>
                <port>8083</port>
                <server>tomcat7</server>
                <path>/</path>
            </configuration>
        </plugin>
    </plugins>
</build>
```

1.6.4 测试服务调用

将聚合工程clean，再install。分别启动服务提供方和服务消费方。使用http://localhost:8083/user/showName.do进行访问：



此时我们也可以再dubbo-admin监控台上面看到我们启动的服务。



1.6.5 工程结构优化

在上面的案例中，我们发现不管是服务提供方还是服务消费方，都定义了重复的代码，那就是我们的服务接口UserService。我们可以将服务接口单独抽取出来，放在一个单独的模块里面，供服务提供方实现，供服务消费方调用。

1.6.5.1 定义服务接口模块

创建dubbo_service_interface。并将服务接口拷贝过来：

```
package com.xq;

public interface UserService {
    public String getName();
}
```

1.6.5.2 改造服务提供方

将UserService接口删除，并在pom.xml里面引入如下依赖：

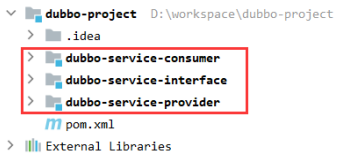
```
<dependencies>
    <dependency>
        <groupId>com.xq</groupId>
        <artifactId>dubbo-service-interface</artifactId>
        <version>1.0-SNAPSHOT</version>
    </dependency>
</dependencies>
```

1.6.5.3 改造服务消费方

将UserService接口删除，并在pom.xml里面引入如下依赖：

```
<dependencies>
    <dependency>
        <groupId>com.xq</groupId>
        <artifactId>dubbo-service-interface</artifactId>
        <version>1.0-SNAPSHOT</version>
    </dependency>
</dependencies>
```

最后工程目录结构如下：



1.7 springboot整合dubbo

1.7.1 创建基础工程

创建基础工程dubbo-service-api。然后引入相关依赖，并创建实体类。

1.7.1.1 引入相关依赖

```
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>5.1.6</version>
    </dependency>
    <dependency>
        <groupId>com.baomidou</groupId>
```

```
        <artifactId>mybatis-plus-boot-starter</artifactId>
        <version>3.5.2</version>
    </dependency>
    <dependency>
        <groupId>com.alibaba</groupId>
        <artifactId>fastjson</artifactId>
        <version>1.2.51</version>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>RELEASE</version>
        <scope>compile</scope>
    </dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <source>7</source>
                <target>7</target>
            </configuration>
        </plugin>
    </plugins>
</build>
```

1.7.1.2 创建实体类

我们需要对用户表进行操作，所以我们创建一个和用户表相关的实体类。注意：实体类一定要实现序列化接口Serializable。

```
@Data
@TableName("tb_user")
public class User implements Serializable {
    @TableId(type = IdType.AUTO)
    private Long uid;
    private String name;
    private Integer age;
    private String email;
    private Integer isDeleted;
    private Integer sex;
}
```

1.7.1.3 创建公共服务接口

公共服务接口，既要被服务提供方实现，也要被服务消费方远程调用。

```
public interface UserService {

    public ResultVO getUserById(Long id);

    public ResultVO saveUser(User user);
}
```

1.7.1.4 创建封装响应结果的实体类

后端向前端响应数据，我们封装成统一的数据格式。

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ResultVO <T> implements Serializable {
    private Integer code;
    private String msg;
    private T data;
    public ResultVO(Integer code,String msg){
        this.code=code;
        this.msg=msg;
    }
}
```

```
package com.xq.utils;

import com.xq.entity.ResultVO;

public class ResultVOUtil {
    public static ResultVO success(){
        return new ResultVO<>(200,"操作成功");
    }
    public static ResultVO success(Object data){
        return new ResultVO<>(200,"操作成功",data);
    }
    public static ResultVO error(Integer code,String msg){
        return new ResultVO<>(code,msg);
    }
}
```

1.7.2 创建服务提供方

1.7.2.1 引入相关依赖

```
<dependencies>
    <dependency>
        <groupId>com.xq</groupId>
        <artifactId>dubbo-service-api</artifactId>
        <version>1.0-SNAPSHOT</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
        <version>2.7.4</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <version>2.7.4</version>
    </dependency>
    <dependency>
        <groupId>com.alibaba.boot</groupId>
        <artifactId>dubbo-spring-boot-starter</artifactId>
        <version>0.2.0</version>
    </dependency>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>5.1.6</version>
    </dependency>
    <dependency>
        <groupId>com.baomidou</groupId>
        <artifactId>mybatis-plus-boot-starter</artifactId>
        <version>3.5.2</version>
    </dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <source>7</source>
                <target>7</target>
```

```
            </configuration>
        </plugin>
    </plugins>
</build>
```

1.7.2.2 创建启动类和配置文件

```
@SpringBootApplication
@EnableDubbo
public class ServiceProviderApplication {
    public static void main(String[] args) {

 SpringApplication.run(ServiceProviderApplication.class,args);
    }
}
```

```
server:
  port: 8081
dubbo:
  application:
    name: dubbo-service-provider
  registry:
    address: zookeeper://127.0.0.1:2181
    protocol: zookeeper
  protocol:
    name: dubbo
    port: 20881
  monitor:
    protocol: registry  # 所有服务配置连接监控中心，进行监控统计
spring:
  datasource:
    username: root
    password: Admin123!
    url: jdbc:mysql://192.168.10.140:3306/dbtest1
    driver-class-name: com.mysql.jdbc.Driver
mybatis-plus:
  mapper-locations: mapper/*.xml
```

1.7.2.3 创建mapper接口

```java
package com.xq.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.entity.User;
import org.apache.ibatis.annotations.Mapper;

@Mapper
public interface UserMapper extends BaseMapper<User> {
}
```

1.7.2.4 创建服务接口实现类

```java
package com.xq.service.impl;

import com.alibaba.dubbo.config.annotation.Service;
import com.xq.entity.ResultVO;
import com.xq.entity.User;
import com.xq.mapper.UserMapper;
import com.xq.service.UserService;
import com.xq.utils.ResultVOUtil;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.HashMap;
import java.util.Map;

@Service // dubbo的service注解
public class UserServiceImpl implements UserService {

    @Autowired
    UserMapper userMapper;

    public ResultVO getUserById(Long id) {
        User user = userMapper.selectById(id);
        return ResultVOUtil.success(user);
    }

    public ResultVO saveUser(User user) {
        int i = userMapper.insert(user);
        if(i<=0){
            return ResultVOUtil.error(400,"添加商品失败");
        }else{
            Map<String,Object> res=new HashMap<>();
            res.put("msg","用户添加成功");
            res.put("id",user.getUid());
```

```
            return ResultVOUtil.success(res);
        }
    }
}
```

1.7.3 创建服务消费方

1.7.3.1引入相关依赖

```
<dependencies>
    <dependency>
        <groupId>com.xq</groupId>
        <artifactId>dubbo-service-api</artifactId>
        <version>1.0-SNAPSHOT</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
        <version>2.7.4</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <version>2.7.4</version>
    </dependency>
    <dependency>
        <groupId>com.alibaba.boot</groupId>
        <artifactId>dubbo-spring-boot-starter</artifactId>
        <version>0.2.0</version>
    </dependency>
</dependencies>
```

1.7.3.2 创建启动类和配置文件

```
@SpringBootApplication
@EnableDubbo
public class ServiceConsumerApplication {
    public static void main(String[] args) {

 SpringApplication.run(ServiceConsumerApplication.class,args);
    }
}
```

```
server:
  port: 8082
```

```yaml
dubbo:
  application:
    name: dubbo-service-provider
  registry:
    address: zookeeper://127.0.0.1:2181
    protocol: zookeeper
  protocol:
    name: dubbo
    port: 20881
  monitor:
    protocol: registry  # 所有服务配置连接监控中心，进行监控统计
spring:
  datasource:
    username: root
    password: Admin123!
    url: jdbc:mysql://192.168.10.140:3306/dbtest1
    driver-class-name: com.mysql.jdbc.Driver
mybatis-plus:
  mapper-locations: mapper/*.xml
```

1.7.3.3 创建控制器

```java
@RestController
public class UserController {

    @Reference // 这里用dubbo提供的@Reference注解实现服务的远程调用
    UserService userService;

    @RequestMapping("/user/findById")
    public ResultVO findById(@RequestParam("id") Long id){
        return userService.getUserById(id);
    }
}
// http://localhost:8082/user/findById?id=1
```

注意在启动服务的时候，先启动服务提供方，再启动服务消费方，否则会报错。

2 dubbo的配置

2.1 dubbo配置--服务启动检查配置

Dubbo缺省会在启动时检查依赖的服务是否可用，不可用时会抛出异常，阻止Spring初始化完成，以便上线时，能及早发现问题，默认check="true”。

可以通过check="false”关闭检查，比如，测试时，有些服务不关心，或者出现了循环依赖，必须有一方先启动。

另外，如果你的Spring容器是懒加载的，或者通过API编程延迟引用服务，请关闭check，否则服务临时不可用时，会抛出异常，拿到null引用，如果check="false”，总是会返回引用，当服务恢复时，能自动连上。

现在我们启动服务消费方，此时会报错：

```
Caused by: java.lang.IllegalStateException Create breakpoint : Failed to check the status of the service com.xq.service.UserService. No provider available for the service
    at com.alibaba.dubbo.config.ReferenceConfig.createProxy(ReferenceConfig.java:422) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.config.ReferenceConfig.init(ReferenceConfig.java:333) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.config.ReferenceConfig.get(ReferenceConfig.java:163) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.config.spring.ReferenceBean.getObject(ReferenceBean.java:66) ~[dubbo-2.6.2.jar:2.6.2]
    ... 30 common frames omitted
```

这是因为dubbo开启了服务启动检查。发现服务提供方没有在注册中心注册，所以报错。

我们只需要在服务提供方和服务消费方的配置文件里面加上如下配置：

```yaml
dubbo:
   consumer:
     check: false # 配置消费者统一检查为false：当服务提供者未启动时，消费者启动不会报错
```

此时再先启动服务消费方则不会报错。如果在xml环境中配置，则配置如下：

```xml
<dubbo:consumer check="false" />
```

2.2 dubbo配置--超时设置

由于网络或服务端不可靠，会导致调用出现一种不确定的中间状态（超时）。为了避免超时导致客户端资源（线程）挂起耗尽，必须设置超时时间。

现在我们改造服务提供方服务接口实现类的代码：

```java
public ResultVO getUserById(Long id) {
    try {
        Thread.sleep( millis: 5000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
    User user = userMapper.selectById(id);
    return ResultVOUtil.success(user);
}
```

模拟超时

此时重新启动服务，通过消费方实现服务的远程调用，此时服务消费端会报超时异常：

```
com.alibaba.dubbo.remoting.TimeoutException Create breakpoint : Waiting server-side response timeout. start time: 2024-06-08 16:05:44.738, end time: 2024-06-08 16:05:45.741,
    at com.alibaba.dubbo.remoting.exchange.support.DefaultFuture.get(DefaultFuture.java:136) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.remoting.exchange.support.DefaultFuture.get(DefaultFuture.java:112) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.dubbo.DubboInvoker.doInvoke(DubboInvoker.java:95) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.AbstractInvoker.invoke(AbstractInvoker.java:148) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.listener.ListenerInvokerWrapper.invoke(ListenerInvokerWrapper.java:77) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.monitor.support.MonitorFilter.invoke(MonitorFilter.java:65) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.ProtocolFilterWrapper$1.invoke(ProtocolFilterWrapper.java:72) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.dubbo.filter.FutureFilter.invoke(FutureFilter.java:54) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.ProtocolFilterWrapper$1.invoke(ProtocolFilterWrapper.java:72) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.filter.ConsumerContextFilter.invoke(ConsumerContextFilter.java:48) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.ProtocolFilterWrapper$1.invoke(ProtocolFilterWrapper.java:72) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.protocol.InvokerWrapper.invoke(InvokerWrapper.java:56) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.support.FailoverClusterInvoker.doInvoke(FailoverClusterInvoker.java:78) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.support.AbstractClusterInvoker.invoke(AbstractClusterInvoker.java:238) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.support.wrapper.MockClusterInvoker.invoke(MockClusterInvoker.java:75) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.proxy.InvokerInvocationHandler.invoke(InvokerInvocationHandler.java:52) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.common.bytecode.proxy0.getUserById(proxy0.java) ~[dubbo-2.6.2.jar:2.6.2]
    at com.xq.controller.UserController.findById(UserController.java:18) ~[classes/:na] <14 internal lines>
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:655) ~[tomcat-embed-core-9.0.65.jar:4.0.FR] <1 internal line>
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:764) ~[tomcat-embed-core-9.0.65.jar:4.0.FR] <33 internal lines>
```

这是因为服务消费方远程调用服务提供方服务的时候，默认等待服务提供方响应时间是1s钟。为了解决问题，我们可以自定义设置超时时间。

关于dubbo超时时间的设置，设置方法有多种，可以设置在**具体方法上**，也可以设置在**具体实现类上**，还能设置在**全局配置**中。这几种又分别都可以设置在**消费者**和**提供者**里面。

2.2.1 在消费端配置超时

2.2.1.1 在消费端配置全局超时

在**服务消费方**作如下设置：

```yaml
dubbo:
  consumer:
    timeout: 6000 # 超时时间的设置要比服务提供方的超时时间长
```

如果在xml环境中配置如下：

```xml
<!--针对于服务提供方中的所有服务接口配置的超时-->
<dubbo:consumer check="false" timeout="6000" />
```

2.2.1.2 在消费端配置服务接口级别的超时

```
@RestController
public class UserController {

    @Reference(timeout = 6000) // 这里用dubbo提供的@Reference注解实现服务的远程调用
    UserService userService;

    @RequestMapping("/user/findById")
    public ResultVO findById(@RequestParam("id") Long id){
        return userService.getUserById(id);
    }
}
```

或者：

```
<!--针对于服务提供方中的某一个服务接口配置的超时-->
<dubbo:reference interface="com.xq.service.UserService"
id="userService" timeout="6000" />
```

2.2.1.3 在消费端配置方法级别的超时

```
@RestController
public class UserController {

    @Reference(parameters = {
            "getUserById.timeout","6000"
    }) // 这里用dubbo提供的@Reference注解实现服务的远程调用
    UserService userService;

    @RequestMapping("/user/findById")
    public ResultVO findById(@RequestParam("id") Long id){
        return userService.getUserById(id);
    }
}
```

还可以精确到某一个方法超时配置：

```
<dubbo:reference interface="com.xq.service.UserService"
id="userService" >
    <dubbo:method name="getUserById" timeout="6000getUserById">
</dubbo:method>
</dubbo:reference>
```

配置超时的优先级： 方法级别超时配置 > 接口超时配置 > 全局超时配置

2.2.2 在服务提供端配置超时

2.2.2.1 配置服务接口级别的超时

```
@Service(timeout = 6000) // dubbo的service注解
public class UserServiceImpl implements UserService {
    ... ...
}
```

2.2.2.2 配置方法级别的超时

```
@Service(parameters = {"getUserById.timeout","6000"}) // dubbo的service注解
public class UserServiceImpl implements UserService {
    ... ...
}
```

但此时直接启动服务会报错：

```
Caused by: org.springframework.beans.ConversionNotSupportedException Create breakpoint : Failed to convert property value of type 'java.lang.String[]' to required type 'java.util.Map
    at org.springframework.beans.AbstractNestablePropertyAccessor.convertIfNecessary(AbstractNestablePropertyAccessor.java:595) ~[spring-beans-5.3.23.jar:5.3.23]
    at org.springframework.beans.AbstractNestablePropertyAccessor.convertForProperty(AbstractNestablePropertyAccessor.java:609) ~[spring-beans-5.3.23.jar:5.3.23]
    at org.springframework.beans.BeanWrapperImpl.convertForProperty(BeanWrapperImpl.java:219) ~[spring-beans-5.3.23.jar:5.3.23]
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.convertForProperty(AbstractAutowireCapableBeanFactory.java:1756) ~[spring-beans-5.3.23.jar:5.3
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.applyPropertyValues(AbstractAutowireCapableBeanFactory.java:1712) ~[spring-beans-5.3.23.jar:5.
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.populateBean(AbstractAutowireCapableBeanFactory.java:1452) ~[spring-beans-5.3.23.jar:5.3.23]
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean(AbstractAutowireCapableBeanFactory.java:619) ~[spring-beans-5.3.23.jar:5.3.23]
    ... 15 common frames omitted
```

原因很明显，@Service里面的parameters是String[]类型，但是要映射到ServiceBean的parameters属性是一个Map<String,String>类型，没法自动转换映射上去,所以报了这个错误。

那么如何解决这个问题呢?

我们查看源码发现ServiceBean是通过:

com.alibaba.dubbo.config.spring.beans.factory.annotation.ServiceAnnotationBeanPostProcessor#registerServiceBean

这个BeanPostProcessor注册进来的，这里面扫描了所有打了@Service的类，把他们注册成了ServiceBean。

于是我们就有了曲线救国的方法，我们在定义一个BeanPostProcessor，在ServiceAnnotationBeanPostProcessor之后执行，然后在ServiceBean真正实例化之前转换一下parameters这个参数为Map<String,String>就好了:

```
package com.xq.processor;

import com.alibaba.dubbo.config.spring.ServiceBean;
```

```
import
com.alibaba.dubbo.config.spring.convert.converter.StringArrayToMa
pConverter;
import
com.alibaba.dubbo.config.spring.convert.converter.StringArrayToSt
ringConverter;
import org.springframework.beans.BeansException;
import org.springframework.beans.PropertyValue;
import org.springframework.beans.PropertyValues;
import
org.springframework.beans.factory.config.InstantiationAwareBeanPo
stProcessorAdapter;
import org.springframework.core.PriorityOrdered;
import org.springframework.core.convert.ConversionService;
import
org.springframework.core.convert.support.DefaultConversionService
;
import org.springframework.stereotype.Component;

import java.beans.PropertyDescriptor;
import java.util.Map;

/**
 * 解决@Service注解配置parameters参数时无法将String[]转化成
Map<String,String>的bug
 */
@Component
public class ServiceParameterBeanPostProcessor extends
InstantiationAwareBeanPostProcessorAdapter implements
PriorityOrdered {
    @Override
    public int getOrder() {
        return PriorityOrdered.LOWEST_PRECEDENCE;
    }

    @Override
    public PropertyValues
postProcessPropertyValues(PropertyValues pvs,
PropertyDescriptor[] pds, Object bean, String beanName) throws
BeansException {
        if (bean instanceof ServiceBean) {
            PropertyValue propertyValue =
pvs.getPropertyValue("parameters");
            ConversionService conversionService =
getConversionService();
```

```
            if (propertyValue != null && propertyValue.getValue()
!= null &&
conversionService.canConvert(propertyValue.getValue().getClass(),
Map.class)) {
                Map parameters =
conversionService.convert(propertyValue.getValue(), Map.class);
                propertyValue.setConvertedValue(parameters);
            }
        }
        return pvs;
    }

    private ConversionService getConversionService() {
        DefaultConversionService conversionService = new
DefaultConversionService();
        conversionService.addConverter(new
StringArrayToStringConverter());
        conversionService.addConverter(new
StringArrayToMapConverter());
        return conversionService;
    }

}
```

此时启动就不会报错。

2.2.2.3 配置全局超时

这里与消费之的区别只是头不同，消费者是：consumer，生产者是：provider

```
dubbo:
  provider:
    timeout: 6000
```

消费者和生产者都配置了超时时间，优先级关系：

> - 在消费者和生产者同级配置中，消费者优先级比生成者优先级高
> - 在消费者和生成者不同级配置中，级别高的优先级高,也就是如下关系:
> - 消费者指定方法 > 生产者指定方法 > 消费者指定服务 > 生产者指定服务 > 消费者全局配置 > 生产者全局配置

2.3 dubbo配置--重试次数

重试次数的配置一般配合服务超时配置一起使用的。当我们远程调用服务失败的时候，我们可以尝试多试几次，如果依然失败，再给客户端响应请求失败的异常信息。

重试次数我们就在服务消费方去配置：

```yaml
dubbo:
  consumer:
    check: false
    timeout: 2000
    retries: 3
```

接下来我们再改造一下服务提供方的业务代码：

```java
@Service
public class UserServiceImpl implements UserService {

    2 usages
    @Autowired
    UserMapper userMapper;

    1 usage
    public ResultVO getUserById(Long id) {
        System.out.println("服务提供方被调用了....");
        try {
            Thread.sleep( millis: 5000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        User user = userMapper.selectById(id);
        return ResultVOUtil.success(user);
    }
```

最后我们启动服务，重新发送请求，观察控制台效果：

```
服务提供方被调用了....
服务提供方被调用了....
服务提供方被调用了....
服务提供方被调用了....
2024-06-13 16:37:33.845 ERROR 26528 --- [dTimer-thread-2] c.a.dubbo.monitor.dubbo.DubboMonitor     :  [DUBBO] Unexpected error occur at send statistic, cause: No provider availab

com.alibaba.dubbo.rpc.RpcException Create breakpoint : No provider available from registry 127.0.0.1:2181 for service com.alibaba.dubbo.monitor.MonitorService on consumer 192.168.10.
    at com.alibaba.dubbo.registry.integration.RegistryDirectory.doList(RegistryDirectory.java:575) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.directory.AbstractDirectory.list(AbstractDirectory.java:74) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.support.AbstractClusterInvoker.list(AbstractClusterInvoker.java:271) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.support.AbstractClusterInvoker.invoke(AbstractClusterInvoker.java:232) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.cluster.support.wrapper.MockClusterInvoker.invoke(MockClusterInvoker.java:75) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.rpc.proxy.InvokerInvocationHandler.invoke(InvokerInvocationHandler.java:52) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.common.bytecode.proxy0.collect(proxy0.java) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.monitor.dubbo.DubboMonitor.send(DubboMonitor.java:112) ~[dubbo-2.6.2.jar:2.6.2]
    at com.alibaba.dubbo.monitor.dubbo.DubboMonitor$1.run(DubboMonitor.java:69) ~[dubbo-2.6.2.jar:2.6.2] <7 internal lines>

2024-06-13 16:38:33.846 ERROR 26528 --- [dTimer-thread-2] c.a.dubbo.monitor.dubbo.DubboMonitor     :  [DUBBO] Unexpected error occur at send statistic, cause: No provider availab
```

第一次请求失败之后，又重试了3次

注意的是，重试配置并不适用于所有操作，只适用于幂等操作，比如(查询、删除、修改)。非幂等操作不能设置重试次数配置，比如新增。

2.4 dubbo配置--多版本

在实际开发中，我们的业务功能需要升级迭代，可能会产生很多个不同的版本，具体要使用哪个版本的服务功能，我们可以通过多版本配置来实现。

第一步：配置服务提供方

在服务提供方创建两个服务实现类：

```
@Service(version = "1.0.0")
public class UserServiceImpl implements UserService {

    2 usages
    @Autowired
    UserMapper userMapper;

    1 usage
    public ResultVO getUserById(Long id) {
        System.out.println("服务提供方1被调用了....");
        try {
            Thread.sleep( millis: 5000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        User user = userMapper.selectById(id);
        return ResultVOUtil.success(user);
    }
```

```
@Service(version = "2.0.0")
public class UserServiceImpl2 implements UserService {

    2 usages
    @Autowired
    UserMapper userMapper;

    1 usage
    public ResultVO getUserById(Long id) {
        System.out.println("服务提供方2被调用了....");
        try {
            Thread.sleep( millis: 5000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        User user = userMapper.selectById(id);
        return ResultVOUtil.success(user);
    }
```

第二步：改造服务消费方的控制器代码

```
@RestController
public class UserController {
    // 这里用dubbo提供的@Reference注解实现服务的远程调用
    /*@Reference(parameters = {
            "getUserById.timeout","6000"
    })*/
    1 usage
    @Reference(version = "2.0.0")
    UserService userService;

    no usages
    @RequestMapping("/user/findById")
    public ResultVO findById(@RequestParam("id") Long id){
        return userService.getUserById(id);
    }
}
```

重启服务，进行测试，我们查看服务提供方控制台：

```
2024-06-13 16:50:15.527  INFO 25796 --- [           main] com.xq.ServiceProviderApplication        : Started ServiceProviderApplication in 4.47 seconds (JVM running for 5.984)
服务提供方2被调用了....
2024-06-13 16:50:44.553  INFO 25796 --- [:20881-thread-2] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2024-06-13 16:50:44.695  INFO 25796 --- [:20881-thread-2] com.zaxxer.hikari.pool.PoolBase          : HikariPool-1 - Driver does not support get/set network timeout for connection
2024-06-13 16:50:44.697  INFO 25796 --- [:20881-thread-2] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2024-06-13 16:50:44.717  INFO 25796 --- [:20881-thread-2] c.a.d.common.concurrent.ExecutionList     :  [DUBBO] Executor for listenablefuture is null, will use default executor!, d
2024-06-13 16:50:44.718  INFO 25796 --- [reator-thread-1] c.a.d.r.zookeeper.ZookeeperRegistry       :  [DUBBO] Register: consumer:///com.alibaba.dubbo.monitor.MonitorService?catego
2024-06-13 16:50:44.757  INFO 25796 --- [reator-thread-1] c.a.d.r.zookeeper.ZookeeperRegistry       :  [DUBBO] Subscribe: consumer:///com.alibaba.dubbo.monitor.MonitorService?cate
2024-06-13 16:50:44.906  INFO 25796 --- [reator-thread-1] c.a.d.r.zookeeper.ZookeeperRegistry       :  [DUBBO] Notify urls for subscribe url consumer:///com.alibaba.dubbo.monitor.
```

我们发现，调用的是version为2.0.0版本的服务。

2.5 dubbo配置--本地存根

在Dubbo中利用本地存根在客户端执行部分逻辑,远程服务后，客户端通常只剩下接口，而实现全在服务器端，但提供方有些时候想在客户端也执行部分逻辑，比如：做ThreadLocal缓存，**提前验证参数**，调用失败后伪造容错数据等等，此时就需要在 API 中带上 Stub，客户端生成 Proxy 实例，会把 Proxy 通过构造函数传给Stub，然后把 Stub 暴露给用户，Stub 可以决定要不要去调 Proxy。

第一步：在服务消费方创建本地存根实现类

```
package com.xq.stub;

import com.xq.entity.ResultVO;
import com.xq.entity.User;
import com.xq.service.UserService;
import com.xq.utils.ResultVOUtil;

/**
 * 本地存根的实现
 */
public class UserServiceStub implements UserService {

    private final UserService userService;

    // 构造函数传入真正的远程代理对象
    public UserServiceStub(UserService userService){
        this.userService = userService;
    }

    public ResultVO getUserById(Long id) {
        System.out.println("本地存根的代码被调用......");
        // 此代码在客户端执行， 你可以在客户端做ThreadLocal本地缓存，或预先验证参数是否合法，等等
        if(id > 0){
            return userService.getUserById(id);
        }
         // 你可以容错，可以做任何AOP拦截事项
        return ResultVOUtil.error(500,"参数非法");
    }

    public ResultVO saveUser(User user) {
        return null;
    }
}
```

第二步：配置本地存根

```yaml
dubbo:
  consumer:
    check: false
    timeout: 6000
    retries: 3
    stub: com.xq.stub.UserServiceStub
```

测试：在发生请求时，我们传递一个小于0的参数。

3 dubbo高可用

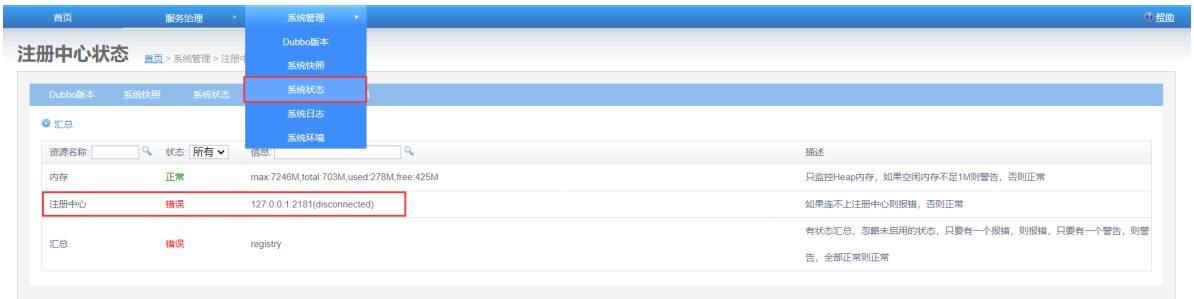
3.1 zookeeper宕机与dubbo直连

结论：zookeeper注册中心宕机，还可以实现服务的远程调用。

原因：

> - 监控中心宕掉不影响使用,只是丢失部分采样数据。
> - 数据库宕掉后，注册中心仍能通过缓存提供服务列表查询，但不能注册新服务
> - 注册中心对等集群，任意一台宕掉后，将自动切换到另一台
> - **注册中心全部宕掉后，服务提供者和服务消费者仍能通过本地缓存通讯**
> - 服务提供者无状态，任意一台宕掉后，不影响使用
> - 服务提供者全部宕掉后，服务消费者应用将无法使用，并无限次重连等待服务提供者恢复

现在我们关闭注册中心，查看系统状态：



此时我们发送请求：

localhost:8082/user/findById?id=1

美观输出


```
{
  "code": 200,
  "msg": "操作成功",
  "data": {
    "uid": 1,
    "name": "Jone",
    "age": 18,
    "email": "jone@baomidou.com",
    "isDeleted": 0,
    "sex": 0
  }
}
```


关闭掉注册中心，服务依然可以被调用

其实我们完全可以绕过注册中心，通过dubbo直连的方式进行服务的远程调用。

```
@RestController
public class UserController {
    // 这里用dubbo提供的@Reference注解实现服务的远程调用
    /*@Reference(parameters = {
            "getUserById.timeout","6000"
    })*/
    1 usage
    @Reference(version = "2.0.0",url = "127.0.0.1:20881")
    UserService userService;
```

dubbo直连的配置

3.2 集群下的dubbo配置

在集群负载均衡时，Dubbo 提供了多种均衡策略，缺省为 random 随机调用。我们可以使用@Reference中的loadbalance属性配置。

```
@RestController
public class UserController {
    // 这里用dubbo提供的@Reference注解实现服务的远程调用
    /*@Reference(parameters = {
            "getUserById.timeout","6000"
    })*/
    1 usage
    @Reference(version = "2.0.0",loadbalance = "roundrobin")
    UserService userService;
```

轮询的配置

dubbo负载均衡策略:

Random LoadBalance	**随机**	**随机，按权重设置随机概率。在一个截面上碰撞的概率高，但调用量越大分布越均匀，而且按概率使用权重后也比较均匀，有利于动态调整提供者权重。**	**random**
RoundRobin LoadBalance	**轮询**	**轮循，按公约后的权重设置轮循比率。存在慢的提供者累积请求的问题，比如：第二台机器很慢，但没挂，当请求调到第二台时就卡在那，久而久之，所有请求都卡在调到第二台上。**	**roundrobin**
LeastActive	**最少活**	**最少活跃调用数，相同活跃数的随机，活跃数指调用前后计**	

LeastActive LoadBalance	活跃优先	能够使慢的提供者收到更少请求，因为越慢的提供者调用前后量数差异越大，而且按概率使用权重后也比较均匀，有利于动态调整提供者权重。	leastactive
Random LoadBalance	随机	随机，按概率设置权重，从统计上看来调用量越多分布越均匀，而且按概率使用权重后也比较均匀，有利于动态调整提供者权重。	random
ConsistentHash LoadBalance	一致性哈希	相同参数的请求总是发到同一提供者。当某一台提供者挂时，原本发往该提供者的请求，基于虚拟节点，平摊到其它提供者，不会引起剧烈变动。	consistenthash

3.3 服务降级

什么是服务降级？

当服务器压力剧增的情况下，根据实际业务情况及流量，对一些服务和页面有策略的不处理或换种简单的方式处理，从而释放服务器资源以保证核心交易正常运作或高效运作。

可以通过服务降级功能临时屏蔽某个出错的非关键服务，并定义降级后的返回策略。

dubbo服务降级的策略有两种：

> - 屏蔽服务 表示不发起远程调用，直接返回null。用来屏蔽不重要服务不可用时对调用方的影响。
> - 服务容错 表示消费方对该服务的方法调用在失败后，再返回 null 值。用来容忍不重要服务不稳定时对调用方的影响。

(1) 屏蔽服务

此时发送请求，页面是空白的。

浏览器页面是空白的，什么也没有，因为我们把服务提供方服务屏蔽了。

(2) 服务容错

改造服务提供方代码：

```java
public ResultVO getUserById(Long id) {
    int i = 10 / 0; //模拟服务调用异常
    System.out.println("服务提供方2被调用了....");
    try {
        Thread.sleep(5000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
    User user = userMapper.selectById(id);
    return ResultVOUtil.success(user);
}
```